

• BOOTCAMP QI TECH 2026 · DIA 1 · TARDE

Entrega final e Base Service

A entrega das próximas quatro semanas e o código de um serviço real.



O ROTEIRO

Duas horas com **três blocos**

13:40

O projeto

O que vocês vão construir até 13 de outubro

14:15

O base-service

O código de um serviço real, camada por camada

15:20

Próximas aulas

O que fazer até sábado que vem

Pergunta pode ser feita a qualquer momento e quem está pelo Meet pergunta pelo chat.

BLOCO 01

01 A entrega

Um protótipo bancário. Em time. Entrega em 13 de outubro.

13:40 às 14:15

A ferramenta é livre · a defesa é obrigatória

A ENTREGA DE VOCÊS:

É construir um banco!

- 01 Cadastro de **cliente**
- 02 **Conta**, vinculada a um cliente
- 03 **Transação**, com controle de saldo
- 04 **Tarifa** na transferência
- 05 **Extrato** paginado

Como cada uma dessas coisas existe no seu sistema é decisão sua e é ela que você vai ter de defender.

A ENTREGA

a construção começa desde **hoje**,
não depois da última aula

Duas peças: a **apresentação** e o **repositório**

1 · A APRESENTAÇÃO DO SISTEMA QUE VOCÊS FIZERAM

Queremos em formato PDF

Slides ou documento, o que vocês preferirem. O que não é livre é a lógica: ela precisa seguir a de uma **RFC**.

- A apresentação é o que **explica o sistema no lugar de vocês**.
- Mostrem o **ecossistema**. Se quebrarem o sistema em mais partes, a apresentação acompanha cada parte no detalhe.
- Expliquem a **regra de negócio**: o que o sistema permite, o que ele barra e por quê.
- Vamos enviar modelos de RFC's para vocês e na próxima aula mostraremos um exemplo de como ela funciona!

2 · O REPOSITÓRIO E COMO A GENTE VAI RODAR

Repositório público no GitHub

Um ou mais repositórios, com o código que vocês escreveram. Pode ficar privado durante a construção, mas precisa estar **público no dia da entrega**.

SOBE COM

docker compose

TESTES COM

pytest

**NA MÁQUINA
DO JURADO****Linux**

AS SUAS ESCOLHAS

O que fica **livre**

Quantas entidades existem

Como controlar o saldo

Como a tarifa é cobrada

Como o dinheiro é representado

Rotas, verbos e formato

Quantidade de testes

De hoje até o **pódio**

13 / 10

Entrega da fase 1

14 e 15/10

A banca avalia as
melhores entregas

16/10

Finalistas divulgados

17 e 18/10

2º fase com outro desafio

13 DE OUTUBRO

**O que estiver no repositório nessa data
é o que a banca vê**

17 E 18/10

**Presencial, contínuo, com pernoite
no escritório**

Um novo desafio que complementa o 1º

Nesses dois dias, cada um traz o próprio computador. Nas
aulas de sábado, não precisa.

O DESAFIO É EM TIME

Do jeito que a QI Tech **constrói produto**

1º LUGAR

R\$ 8.000

para o time

2º LUGAR

R\$ 4.000

para o time

3º LUGAR

R\$ 3.000

para o time

O time é formado por quem está no Bootcamp. **Um time que não combina como divide o trabalho perde mais tempo do que ganha.**

A ENTREGA



BLOCO 02

02 o base-service

O código de um serviço de verdade, camada por camada. O que cada uma decide e o que ela não pode decidir.

bootcamp-base-api

Uma API REST em Python + FastAPI

É a primeira linha do README dele. Todo código que aparecer nesta tela está nesse repositório, no arquivo e na linha que eu vou mostrar.

Postgres e Docker

Sobe na sua máquina com um comando. Nada de ambiente mágico: o que roda aqui roda aí.

Testes

Testes que cobrem todos os fluxos e todas as camadas.

Ele é público, e é de vocês. **As camadas que vocês vão ver aqui são as mesmas com que a gente constrói de verdade.**

DO LADO DE FORA ATÉ O BANCO E DE VOLTA

A requisição, **camada por camada**

IDA · O PEDIDO

app.py

middlewares

schemas

dtos

resource

controllers

repositories

models

database.sql

VOLTA · A RESPOSTA

O BASE-SERVICE

Um **restaurante**, do pedido ao prato

IDA

app.py → middleware → schema → resource →
controller → repository → models → banco

VOLTA

banco → models → repository → controller →
resource → DTO → middleware

ANALOGIA GERAL

Um restaurante. O pedido do cliente entra pela porta, passa por várias pessoas com funções bem definidas, chega à despensa e volta como um prato pronto. Ninguém faz o trabalho do outro. Se o garçom começa a cozinhar, o restaurante quebra.

POR QUE DIVIDIR EM CAMADAS

Cada camada responde a uma pergunta só. Quando algo quebra, você sabe em qual arquivo olhar. Quando algo muda, você sabe qual arquivo mexer, e só ele.

app.py: o ponto de partida

- src/app.py

O QUE É

O ponto de partida. Monta a aplicação peça por peça: registra os middlewares, liga cada endereço HTTP a quem atende.

ANALOGIA

O dono abrindo o restaurante de manhã. Escala quem fica na porta, pendura o cardápio e define quem atende cada mesa. Ele não cozinha nem serve.

REGRA DE OURO

É a lista completa do que a API atende. Rota que não está aqui não existe. E ele diz quem atende cada endereço, nunca como a resposta sai.

```
def create_app() -> FastAPI:

    application = FastAPI(
        docs_url=None,
        redoc_url=None,
        openapi_url=None,
    )

    register_session_manager_middleware(application)
    register_internal_token_middleware(application)
    register_request_logger_middleware(application)
    register_request_context_middleware(application)

    sample_entity_resource = SampleEntityResource()

    application.add_api_route(
        "/sample_entity",
        sample_entity_resource.on_post,
        methods=["POST"],
    )
    application.add_api_route(
        "/sample_entity/{sample_entity_key}",
        sample_entity_resource.on_get_by_key,
        methods=["GET"],
    )
```

Middleware: cascas de cebola

O QUE É

Camadas por fora da aplicação. Toda requisição atravessa todas na ida e todas de novo na volta. Neste projeto são quatro: identificador único, log de entrada e saída, verificação do token e ciclo de vida da sessão de banco.

ANALOGIA

A recepcionista do restaurante, que confere se você tem reservas. O sistema de segurança do restaurante para garantir nenhum roubo. A regra de vestimenta do restaurante.

DUAS IDEIAS-CHAVE

A ordem importa e é regra, não gosto: log antes do token, para registrar tentativas recusadas; banco por dentro de tudo, porque é o recurso mais caro. O middleware de sessão faz **rollback** se deu errado e **close** sempre, mas nunca commit.

- `register_***_middleware`

```
def create_app() -> FastAPI:

    application = FastAPI(
        docs_url=None,
        redoc_url=None,
        openapi_url=None,
    )

    register_session_manager_middleware(application)
    register_internal_token_middleware(application)
    register_request_logger_middleware(application)
    register_request_context_middleware(application)

    sample_entity_resource = SampleEntityResource()

    application.add_api_route(
        "/sample_entity",
        sample_entity_resource.on_post,
        methods=["POST"],
    )
    application.add_api_route(
        "/sample_entity/{sample_entity_key}",
        sample_entity_resource.on_get_by_key,
        methods=["GET"],
    )
```

Um **middleware**, linha por linha

```
> middlewares > internal_token.py > ...
Gilberto.oliveira, 3 weeks ago | 1 author (Gilberto.oliveira)
1 from fastapi import FastAPI, Request
2
3 from constants import BYPASS_ENDPOINTS, INTERNAL_TOKEN
4 from errors.base_error import ForbiddenNotInternal
5 from errors.handlers import qi_exception_to_response
6
7
8 def register_internal_token_middleware(application: FastAPI) -> None:
9     """Deixa passar só quem manda o header INTERNAL-TOKEN com o valor certo.
10
11     O valor certo vem da variável de ambiente INTERNAL_TOKEN, nunca do
12     código. As rotas de BYPASS_ENDPOINTS (raiz e health check) são públicas.
13
14     Repare que aqui a gente DEVOLVE a resposta de erro em vez de levantar
15     a exceção: dentro de um middleware, ninguém está escutando pra traduzir
16     o erro – quem traduz são os exception handlers, que rodam mais pra
17     dentro. Levantar aqui viraria um 500.
18     """
19
20     @application.middleware("http")
21     async def check_internal_token(request: Request, call_next):
22         is_public = request.url.path in BYPASS_ENDPOINTS
23
24         if request.method == "OPTIONS" or is_public:
25             return await call_next(request)
26
27         if request.headers.get("INTERNAL-TOKEN") != INTERNAL_TOKEN:
28             return qi_exception_to_response(ForbiddenNotInternal())
29
30         return await call_next(request)
```

Repare que aqui a gente **devolve** a resposta de erro em vez de levantar a exceção: dentro de um middleware ninguém está escutando para traduzir o erro.

Schema: o contrato de entrada

O QUE É

Um arquivo .json (JSON Schema) que descreve o formato aceito no corpo da requisição: quais campos, de que tipo, obrigatórios ou não, com qual padrão. Um decorator confere o corpo antes do resource rodar.

ANALOGIA

A comanda pré-impressa. Tem campos fixos, e o garçom devolve se você deixou um em branco ou escreveu um número onde ia texto. Ele não confere se o prato existe ou se o restaurante tem o ingrediente. Isso é da cozinha.

REGRA DE OURO

Schema cobra **formato, não verdade**. Garante que a data tem quatro dígitos, traço, dois, traço, dois. Não sabe quantos dias fevereiro tem. Por isso o contrato mora num arquivo que quem integra lê sem abrir o código.

- `@SchemaHandler.validate("post_sample_entity.json")`

```
src > resources > sample_entity.py > SampleEntityResource > on_post
7  from controllers import SampleEntityController
8  from utils.schema_handler import SchemaHandler
9
10
11  class SampleEntityResource:
12      @SchemaHandler.validate("post_sample_entity.json")
13      def on_post(self, payload: dict) -> JSONResponse:
14          controller = SampleEntityController()
15          sample_entity = controller.create(payload)
16
17          return JSONResponse(
18              content=jsonable_encoder(sample_entity),
19              status_code=status.HTTP_201_CREATED,
20          )
21
```

Um **schema.json** de verdade

```
src > schemas > {} post_sample_entity.json > ...
Gilberto.oliveira, 41 minutes ago | 1 author (Gilberto.oliveira)

1  {
2    "title": "PostSampleEntity",
3    "type": "object",
4    "properties": {
5      "hello": {
6        "type": "string",
7        "minLength": 1,
8        "maxLength": 255
9      },
10     "name": {
11       "type": "string",
12       "minLength": 1,
13       "maxLength": 255
14     },
15     "email": {
16       "type": "string",
17       "maxLength": 255,
18       "pattern": "^[^@\\s]+@[^@\\s]+\\.\\.[^@\\s]+$"
19     },
20     "document_number": {
21       "type": "string",
22       "pattern": "^\\d{3}\\.\\.\\d{3}\\.\\.\\d{3}-\\d{2}$"
23     },
24     "birthdate": {
25       "type": "string",
26       "pattern": "^\\d{4}-\\d{2}-\\d{2}$"
27     }
28   },
29   "required": [
30     "hello",
31     "name",
32     "email",
33     "document_number",
34     "birthdate"
35   ],
36   "additionalProperties": false
37 }
38
```

Campos, tipos, tamanhos, padrões e a lista de obrigatórios. `additionalProperties: false`: o que não está no contrato não entra.

Resource: a porta de entrada HTTP

- `application.add_api_route("/sample_entity", ...)`

O QUE É

A porta de entrada HTTP. Faz três coisas, nesta ordem: confere o corpo (via schema), chama o controller, devolve o que o controller respondeu, com o status HTTP certo (201, 202, 204, 200).

ANALOGIA

O garçom. Anota o pedido, leva à cozinha e traz o prato. Não decide se o cliente pode pedir vinho, não vai à despensa, não tempera nada.

REGRA DE OURO

Um método daqui cabe em três linhas. Se não couber, uma regra de negócio vazou da camada de baixo. Aqui não tem if sobre o estado da entidade nem SQL. Mas o status da resposta é decidido aqui, porque só o resource fala a língua do HTTP.

```
def create_app() -> FastAPI:

    application = FastAPI(
        docs_url=None,
        redoc_url=None,
        openapi_url=None,
    )

    register_session_manager_middleware(application)
    register_internal_token_middleware(application)
    register_request_logger_middleware(application)
    register_request_context_middleware(application)

    sample_entity_resource = SampleEntityResource()

    application.add_api_route(
        "/sample_entity",
        sample_entity_resource.on_post,
        methods=["POST"],
    )
    application.add_api_route(
        "/sample_entity/{sample_entity_key}",
        sample_entity_resource.on_get_by_key,
        methods=["GET"],
    )
```

Um **resource** em três linhas

```
src > resources > sample_entity.py > SampleEntityResource > on_post
7  from controllers import SampleEntityController
8  from utils.schema_handler import SchemaHandler
9
10
11  ...
11  class SampleEntityResource:
12  |   @SchemaHandler.validate("post_sample_entity.json")
13  |   def on_post(self, payload: dict) -> JSONResponse:
14  |       controller = SampleEntityController()
15  |       sample_entity = controller.create(payload)
16  |
17  |       return JSONResponse(
18  |           content=jsonable_encoder(sample_entity),
19  |           status_code=status.HTTP_201_CREATED,
20  |       )
21
```

Decorator, chamada ao controller, resposta com HTTP_201_CREATED. Nada além disso.

O CHEF

Controller: onde mora o pode e o não pode

O QUE É

As regras de negócio. É ele que pergunta ao repository o que existe no mundo, decide o que fazer, e dá o commit quando o trabalho terminou.

ANALOGIA

O chef. Recebe a comanda já legível, confere se tem ingrediente, se o cliente é maior de idade para o vinho, se aquele prato ainda está no cardápio. Depois manda o despenseiro buscar e guardar. Só ele diz "pode servir".

DUAS IDEIAS-CHAVE

As perguntas são sobre o que só se sabe olhando o mundo: CPF válido? Alguém já usou este e-mail? Que idade a pessoa tem hoje? Toda checagem acontece **antes** de qualquer escrita, e o commit é a última linha antes do return.

- `sample_entity = controller.create(payload)`

```
src > resources > sample_entity.py > SampleEntityResource > on_post
7   from controllers import SampleEntityController
8   from utils.schema_handler import SchemaHandler
9
10
11  ...
11  class SampleEntityResource:
12      @SchemaHandler.validate("post_sample_entity.json")
13      def on_post(self, payload: dict) -> JSONResponse:
14          controller = SampleEntityController()
15          sample_entity = controller.create(payload)
16
17          return JSONResponse(
18              content=jsonable_encoder(sample_entity),
19              status_code=status.HTTP_201_CREATED,
20          )
21
```

Um **controller** decidindo

```
Gilberto.oliveira, 28 minutes ago | 1 author (Gilberto.oliveira)
22 class SampleEntityController(BaseController):
23     """As regras de negócio. Aqui mora o "pode" e o "não pode"."""
24
25     def __init__(self) -> None:
26         super().__init__(__name__)
27         self.sample_entity_repository = SampleEntityRepository(self.context)
28
29     def create(self, sample_entity_data: dict) -> dict:
30         self.logger.debug("Criando uma nova Sample Entity")
31
32         document_number = sample_entity_data["document_number"]
33         email = sample_entity_data["email"]
34         birthdate = self._parse_birthdate(sample_entity_data["birthdate"])
35
36         if not is_valid_cpf(document_number):
37             raise InvalidDocumentNumber(document_number)
38
39         if self.sample_entity_repository.get_by_document_number(document_number) is not None:
40             raise DuplicatedDocumentNumber(document_number)
41
42         if self.sample_entity_repository.get_by_email(email) is not None:
43             raise DuplicatedEmail(email)
44
45         age = self._age_in_years(birthdate)
46
47         if age < MINIMUM_AGE:
48             raise UnderageSampleEntity(age, MINIMUM_AGE)
49
50         sample_entity = self.sample_entity_repository.create(sample_entity_data=sample_entity_data)
51         if age > MAXIMUM_AGE:
52             new_status = SampleEntityStatus.FAILED
53         else:
54             new_status = SampleEntityStatus.PENDING
55
56         self.sample_entity_repository.update_status(sample_entity, new_status)
57
58         sample_entity_dto = SampleEntityDTO.only_obj_key(sample_entity)
59         self.session.commit()
60
61         return sample_entity_dto
62
```

CPF, e-mail duplicado,
idade mínima e máxima. As
recusas vêm todas antes
do create, e o commit
fecha o trabalho.

Repository: a única porta do banco

O QUE É

A única camada que fala com o banco. Só aqui existe query. Busca, guarda e atualiza. Não decide nada.

ANALOGIA

O despenseiro. É o único com a chave da despensa. O chef pede "traz o cliente de e-mail tal", e ele traz ou diz que não tem. Ele não opina se o prato deve ser feito.

REGRA DE OURO

Nenhuma regra de negócio mora aqui. Se um dia o banco trocar de tecnologia, este é o único arquivo que muda. Ele recebe o contexto da requisição e tira a sessão de lá, em vez de receber a sessão solta.

- `self.sample_entity_repository.get_by_document_number(...)`

```
Gilberto.oliveira, 28 minutes ago | 1 author (Gilberto.oliveira)
22 class SampleEntityController(BaseController):
23     """As regras de negócio. Aqui mora o "pode" e o "não pode"."""
24
25     def __init__(self) -> None:
26         super().__init__(name_)
27         self.sample_entity_repository = SampleEntityRepository(self.context)
28
29     def create(self, sample_entity_data: dict) -> dict:
30         self.logger.debug("Criando uma nova Sample Entity")
31
32         document_number = sample_entity_data["document_number"]
33         email = sample_entity_data["email"]
34         birthdate = self._parse_birthdate(sample_entity_data["birthdate"])
35
36         if not is_valid_cpf(document_number):
37             raise InvalidDocumentNumber(document_number)
38
39         if self.sample_entity_repository.get_by_document_number(document_number) is not None:
40             raise DuplicatedDocumentNumber(document_number)
41
42         if self.sample_entity_repository.get_by_email(email) is not None:
43             raise DuplicatedEmail(email)
44
45         age = self._age_in_years(birthdate)
46
47         if age < MINIMUM_AGE:
48             raise UnderageSampleEntity(age, MINIMUM_AGE)
49
50         sample_entity = self.sample_entity_repository.create(sample_entity_data=sample_entity_data)
51         if age > MAXIMUM_AGE:
52             new_status = SampleEntityStatus.FAILED
53         else:
54             new_status = SampleEntityStatus.PENDING
55
56         self.sample_entity_repository.update_status(sample_entity, new_status)
57
58         sample_entity_dto = SampleEntityDTO.only_obj_key(sample_entity)
59         self.session.commit()
60
61         return sample_entity_dto
62
```

Um exemplo de **repository**

```
src > repositories > sample_entity_repository.py > SampleEntityRepository > update_status
Gilberto.oliveira, 44 minutes ago | 2 authors (Gilberto.oliveira and one other)
1 from datetime import date, datetime
2 from uuid import uuid4
3
4 from database import Context
5 from models import SampleEntity, SampleEntityStatus, SampleEntityStatusEvent
6
7
8 Gilberto.oliveira, 44 minutes ago | 2 authors (Gilberto.oliveira and one other)
9 class SampleEntityRepository:
10
11     def __init__(self, context: Context) -> None:
12         self.session = context.db_session
13
14     def create(self, sample_entity_data: dict) -> SampleEntity:
15         sample_entity = SampleEntity()
16
17         sample_entity.sample_entity_data = {"hello": sample_entity_data["hello"]}
18         sample_entity.name = sample_entity_data["name"]
19         sample_entity.email = sample_entity_data["email"]
20         sample_entity.document_number = sample_entity_data["document_number"]
21         sample_entity.birthdate = date.fromisoformat(sample_entity_data["birthdate"])
22         sample_entity.sample_entity_key = str(uuid4())
23         sample_entity.counter = 0
24         sample_entity.status = self.get_status(SampleEntityStatus.CREATED)
25
26         self.session.add(sample_entity)
27         return sample_entity
28
29     def update_status(self, sample_entity: SampleEntity, new_status_enum: str) -> None:
30         new_status = self.get_status(new_status_enum)
31         sample_entity.status = new_status
32
33         new_status_event = SampleEntityStatusEvent()
34         new_status_event.status = new_status
35         new_status_event.event_datetime = datetime.now()
36
37         sample_entity.status_events.append(new_status_event)
38
39     def get_by_key(self, sample_entity_key: str) -> SampleEntity:
40         return self.session.query(SampleEntity).filter(SampleEntity.sample_entity_key == sample_entity_key).first()
41
42     def get_by_document_number(self, document_number: str) -> SampleEntity:
43         return self.session.query(SampleEntity).filter(SampleEntity.document_number == document_number).first()
```

Monta o objeto, adiciona na sessão, atualiza status e busca por chave, documento ou e-mail. Todo o SQL do projeto vive em arquivos como este.

Models: o espelho da tabela

O QUE É

Classes Python que espelham as tabelas do banco, coluna por coluna, com tipos, chaves estrangeiras e restrições de unicidade. É o ORM (SQLAlchemy) traduzindo linha de tabela em objeto.

ANALOGIA

O mapa da despensa. Diz que na prateleira "sample_entity" existem as colunas nome, e-mail, documento, e que o status é um número apontando para outra prateleira. É a descrição de como as coisas estão guardadas, não de como elas são servidas.

REGRA DE OURO

Model descreve a tabela. Ele carrega detalhes internos (id numérico, coluna JSON, chave estrangeira) que o cliente da API nunca deveria ver. Por isso ele não sai direto na resposta.

- `SampleEntity()` • `self.session.query(SampleEntity)`

```
Gilberto.oliveira, 44 minutes ago | 2 authors (Gilberto.oliveira and one other)
8 class SampleEntityRepository:
9
10     def __init__(self, context: Context) -> None:
11         self.session = context.db_session
12
13     def create(self, sample_entity_data: dict) -> SampleEntity:
14         sample_entity = SampleEntity()
15
16         sample_entity.sample_entity_data = {"hello": sample_entity_data["hello"]}
17         sample_entity.name = sample_entity_data["name"]
18         sample_entity.email = sample_entity_data["email"]
19         sample_entity.document_number = sample_entity_data["document_number"]
20         sample_entity.birthdate = date.fromisoformat(sample_entity_data["birthdate"])
21         sample_entity.sample_entity_key = str(uuid4())
22         sample_entity.counter = 0
23         sample_entity.status = self.get_status(SampleEntityStatus.CREATED)
24
25         self.session.add(sample_entity)
26         return sample_entity
27
28     def update_status(self, sample_entity: SampleEntity, new_status_enum: str) -> None:
29         new_status = self.get_status(new_status_enum)
30         sample_entity.status = new_status
31
32         new_status_event = SampleEntityStatusEvent()
33         new_status_event.status = new_status
34         new_status_event.event_datetime = datetime.now()
35
36         sample_entity.status_events.append(new_status_event)
37
38     def get_by_key(self, sample_entity_key: str) -> SampleEntity:
39         return self.session.query(SampleEntity).filter(SampleEntity.sample_entity_key == sample_entity_key).first()
40
41     def get_by_document_number(self, document_number: str) -> SampleEntity:
42         return self.session.query(SampleEntity).filter(SampleEntity.document_number == document_number).first()
43
```

Um **model**, coluna por coluna

models/sample_entity.py

```
Gilberto.oliveira, 1 hour ago | 1 author (Gilberto.oliveira)
8 class SampleEntity(Base):
9     __tablename__ = "sample_entity"
10
11     id = Column(Integer, primary_key=True)
12     sample_entity_key = Column(CHAR(36), nullable=False)
13     sample_entity_data = Column(JSONB, nullable=False)
14     name = Column(String(255), nullable=False)
15     email = Column(String(255), nullable=False)
16     document_number = Column(CHAR(14), nullable=False)
17     birthdate = Column(Date, nullable=False)
18     counter = Column(Integer, nullable=False)
19     status_id = Column(Integer, ForeignKey(SampleEntityStatus.id), nullable=False)
20     created_at = Column(DateTime, nullable=False, server_default=func.now())
21
22     __table_args__ = (
23         UniqueConstraint("sample_entity_key"),
24         UniqueConstraint("document_number"),
25         UniqueConstraint("email"),
26     )
27
28     status = relationship("SampleEntityStatus", foreign_keys=[status_id], lazy="selectin")
29
30     status_events = relationship(
31         "SampleEntityStatusEvent",
32         back_populates="sample_entity",
33         order_by="asc(SampleEntityStatusEvent.event_datetime)",
34     )
35
```

a tabela no banco

```
14 CREATE TABLE sample_entity(
15     id SERIAL PRIMARY KEY,
16     sample_entity_key CHAR(36) NOT NULL,
17     status_id INTEGER NOT NULL REFERENCES sample_entity_status(id),
18     sample_entity_data JSONB NOT NULL,
19     name VARCHAR(255) NOT NULL,
20     email VARCHAR(255) NOT NULL,
21     document_number CHAR(14) NOT NULL,
22     birthdate DATE NOT NULL,
23     counter INTEGER NOT NULL,
24     created_at TIMESTAMP NOT NULL DEFAULT(NOW()),
25     UNIQUE(sample_entity_key),
26     UNIQUE(document_number),
27     UNIQUE(email)
28 );
29
```

Tipos, nullable, chave estrangeira de status, restrições de unicidade e os relacionamentos. É a tabela descrita em Python.

Banco de dados: onde o dado vive de verdade

O QUE É

Onde o dado vive de verdade e sobrevive ao desligar da API. A conversa com ele acontece por uma sessão: um rascunho onde as mudanças ficam até alguém mandar salvar (commit) ou rasgar (rollback).

ANALOGIA

A despesa. Tudo que entra e sai passa pelo despenseiro. A sessão é o carrinho: você vai enchendo, e só na hora do "pode servir" o que está nele vira estoque de fato. Se der problema no meio, o carrinho volta vazio.

REGRA DE OURO

Conexão emprestada tem de ser devolvida sempre. Conexão que não volta é conexão que falta para a próxima requisição.

É assim que uma API para de responder sem ninguém entender por quê.

commit

vira estoque de fato

rollback

o carrinho volta vazio

DTO: o que chega à mesa

O QUE É

Data Transfer Object. Traduz o objeto do banco (o model) no dicionário simples que vira o JSON da resposta. Escolhe o que sai, com que nome e em que formato.

ANALOGIA

O prato como chega à mesa. Ninguém serve a panela. O cliente recebe a comida bonita, sem a caixa de ingredientes, sem o código da prateleira. O hello, que no banco vive dentro de uma coluna JSON, aqui vira um campo plano com nome de gente.

REGRA DE OURO

Campo novo na resposta se acrescenta aqui, e só aqui. O DTO faz uma cópia antes de mexer, para não sujar o objeto do banco que o resto da requisição ainda vai usar.

- `SampleEntityDTO.only_obj_key(sample_entity)`

```
Gilberto.oliveira, 28 minutes ago | 1 author (Gilberto.oliveira)
22 class SampleEntityController(BaseController):
23     """As regras de negócio. Aqui mora o "pode" e o "não pode"."""
24
25     def __init__(self) -> None:
26         super().__init__(__name__)
27         self.sample_entity_repository = SampleEntityRepository(self.context)
28
29     def create(self, sample_entity_data: dict) -> dict:
30         self.logger.debug("Criando uma nova Sample Entity")
31
32         document_number = sample_entity_data["document_number"]
33         email = sample_entity_data["email"]
34         birthdate = self._parse_birthdate(sample_entity_data["birthdate"])
35
36         if not is_valid_cpf(document_number):
37             raise InvalidDocumentNumber(document_number)
38
39         if self.sample_entity_repository.get_by_document_number(document_number) is not None:
40             raise DuplicatedDocumentNumber(document_number)
41
42         if self.sample_entity_repository.get_by_email(email) is not None:
43             raise DuplicatedEmail(email)
44
45         age = self._age_in_years(birthdate)
46
47         if age < MINIMUM_AGE:
48             raise UnderageSampleEntity(age, MINIMUM_AGE)
49
50         sample_entity = self.sample_entity_repository.create(sample_entity_data=sample_entity_data)
51         if age > MAXIMUM_AGE:
52             new_status = SampleEntityStatus.FAILED
53         else:
54             new_status = SampleEntityStatus.PENDING
55
56         self.sample_entity_repository.update_status(sample_entity, new_status)
57
58         sample_entity_dto = SampleEntityDTO.only_obj_key(sample_entity)
59         self.session.commit()
60
61         return sample_entity_dto
62
```

Um **DTO** escolhendo o que sai

```
Gilberto.oliveira, 1 hour ago | 1 author (Gilberto.oliveira)
class SampleEntityDTO:
    @staticmethod
    def obj_to_dict(sample_entity: SampleEntity) -> dict:
        sample_entity_dto = SampleEntityDTO.obj_to_simplified_dict(sample_entity)
        sample_entity_dto["status_events"] = []

        for status_event in sample_entity.status_events:
            status_event_dto = dict()
            status_event_dto["status"] = status_event.status.enumerator
            status_event_dto["event_datetime"] = status_event.event_datetime.isoformat()

            sample_entity_dto["status_events"].append(status_event_dto)

        return sample_entity_dto

    @staticmethod
    def obj_to_simplified_dict(sample_entity: SampleEntity) -> dict:
        dto = deepcopy(sample_entity.sample_entity_data)
        dto["sample_entity_key"] = sample_entity.sample_entity_key
        dto["name"] = sample_entity.name
        dto["email"] = sample_entity.email
        dto["document_number"] = sample_entity.document_number
        dto["birthdate"] = sample_entity.birthdate.isoformat()
        dto["status"] = sample_entity.status.enumerator
        dto["counter"] = sample_entity.counter

        return dto

    @staticmethod
    def list_obj_to_list_dict(sample_entities_list: List[SampleEntity]) -> List[dict]:
        sample_entities_dict_list = []

        for sample_entity in sample_entities_list:
            sample_entities_dict_list.append(
                SampleEntityDTO.obj_to_simplified_dict(sample_entity)
            )

        return sample_entities_dict_list

    @staticmethod
    def only_obj_key(sample_entity: SampleEntity) -> dict:
        dto = dict()
        dto["sample_entity_key"] = sample_entity.sample_entity_key

        return dto
```

deepcopy antes de mexer, campos planos com nome de gente e o only_obj_key para quando a resposta é só a chave.

DO LADO DE FORA ATÉ O BANCO E DE VOLTA

A requisição, **camada por camada**

IDA · O PEDIDO

app.py

middlewares

schemas

dtos

resource

controllers

repositories

models

database.sql

VOLTA · A RESPOSTA

O BASE-SERVICE

15:20 · A PONTE

Até sábado que vem

Forme o time

Quem já veio com time, está resolvido. Quem não, fala com a gente que ajudamos a formar um time até 3 pessoas.

Clone e suba o repositório

A primeira coisa que pode dar errado é o Docker na sua máquina. Descubra isso hoje, não no dia 12 de outubro.

Se prepare e avance no projeto

Durante as semanas, o ideal é chegar na próxima aula já tendo passado o olho no material de rotas e contratos, banco de dados e fluxos. Tudo fica disponível no Slack.

Próxima aula

O assunto é **modelagem**: banco de dados, ACID e mais.

Na próxima aula, **19 de setembro**, o assunto é modelagem e é ela que decide onde o saldo mora.

QUANDO TRAVAR

Onde pedir **socorro**

Monitoria, terça e quinta

Outros Tech Leads e desenvolvedores para ajudar vocês na entrega e nas outras dúvidas.

HORÁRIO

19h às 20h

LINK

Sala do Meet disponibilizada pelo Slack

O canal do Slack

Onde toda a comunicação acontece desde o prazos, conteúdos para as próximas aulas, dúvidas, networking etc.

NÃO ENTROU AINDA?

Fala com @Sarah e @Pedro

Obrigado!

Bora pra cima! #tmj

Dúvidas?

